

Kubernetes and Cloud Native Associate - KCNA

Scheduling

Manual Scheduling

Welcome to this lesson on manually scheduling pods on a node. In this guide, we explore methods for assigning pods to nodes without relying on Kubernetes' built-in scheduler, which can help in scenarios where you need greater control over pod placement.

Understanding Pod Scheduling

When you create a pod, its manifest typically contains a field called `nodeName`. By default, this field is left unset, allowing the Kubernetes scheduler to assign the pod automatically. Consider the following manifest:

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    name: nginx
spec:
  containers:
    - name: nginx
      image: nginx
      ports:
```

```
- containerPort: 8080
nodeName:
```

Under normal circumstances, the Kubernetes scheduler scans for pods without a specified `nodeName`, determines the appropriate node based on its scheduling algorithm, and creates a binding object to assign the pod to that node.



Note

Without an active scheduler, pods will remain in the **Pending** state. You can confirm this by executing:

```
kubect1 get pods
# Output:
# NAME      READY   STATUS    RESTARTS   AGE
# nginx     0/1     Pending   0           3s
```

Approaches to Manual Pod Scheduling

There are two primary methods for manually scheduling a pod:

1. Specify `nodeName` During Pod Creation

The simplest approach is to set the `nodeName` in the pod's specification. When this field is explicitly defined, Kubernetes assigns the pod immediately to the designated node. See the updated manifest example below:

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
```

```
    name: nginx
spec:
  containers:
    - name: nginx
      image: nginx
      ports:
        - containerPort: 8080
  nodeName: node02
```

2. Scheduling an Existing Pod Using a Binding Object

If the pod is already created and its `nodeName` cannot be modified, you need to simulate the scheduler's behavior by creating a Binding object. This involves the following two steps:

1. Create the Binding Object

Define a binding object that specifies the target node:

```
apiVersion: v1
kind: Binding
metadata:
  name: nginx
target:
  apiVersion: v1
  kind: Node
  name: node02
```

2. Send the Binding Object via a POST Request

Convert the YAML definition into JSON and use a `curl` command to send a POST request to the pod's binding API:

```
curl --header "Content-Type: application/json" \
  --request POST \
  --data '{"apiVersion":"v1", "kind": "Binding", "metadata": {"name": "ngi
http://$SERVER/api/v1/namespaces/default/pods/$PODNAME/binding/
```



Tip

Ensure that you replace ``\$SERVER`` and ``\$PODNAME`` with your actual server address and pod name.

Summary

In summary, you have two options for manually scheduling a pod in Kubernetes:

- **During Pod Creation:** Set the ``nodeName`` field in your pod's manifest to assign it directly to a node.
- **For Existing Pods:** Create a Binding object and use a POST request to assign the pod to your desired node.

This approach provides flexibility in environments where automated scheduling may not fit specific use cases. For more detailed information on Kubernetes pod management, visit the [Kubernetes Documentation](#).

Happy scheduling!



Watch Video



Beta



● Search docs...



Kubernetes and Cloud Native Associate - KCNA ✓